Universal Verification Methodology (UVM) – 50 Important Questions & Answers

---

1. Why do we need UVM methodology?

UVM is a standard way to test digital designs. It helps build reusable and organized testbenches, provides ready-made base classes, and ensures scalability, reusability, and standardization.

2. Why are phases introduced in UVM? What are the phases?

Phases synchronize dynamic objects/components in UVM. Phases: Build, Connect, End-of-Elaboration, Start-of-Simulation, Run, Extract, Check, Report, Final.

3. Difference between uvm_component and uvm_object:

uvm_component: Static, lives throughout simulation, constructor: name and parent.

uvm_object: Dynamic, may live shorter, constructor: name only.

4. Difference between create() and new():

new() is a constructor for SystemVerilog classes. create() uses the UVM factory mechanism to instantiate uvm_object or uvm_component classes and supports factory overrides.

5. What is severity and verbosity in UVM?

Severity controls the importance of log messages: UVM_INFO, UVM_WARNING, UVM_ERROR, UVM_FATAL.

Verbosity controls message detail: UVM_NONE, UVM_LOW, UVM_MEDIUM, UVM_HIGH, UVM_FULL, UVM_DEBUG.

6. Difference between copy() and clone():

copy() performs a deep copy into an existing object. clone() creates a new object and performs copy.

7. How to run any test case in UVM:

run_test("test_name"); triggers all UVM phases from build to run, ending with $finish.

8. What approach does build_phase use and why:

Top-down approach: parent components are built first, ensuring proper hierarchy.

9. Which UVM phases take simulation time:

Run Phase, Reset (Pre/Post), Configure (Pre/Post), Main (Pre/Post), Shutdown (Pre/Post).

10. Bottom-up and top-down approaches in UVM phases:

Top-down: higher-level components processed first (build_phase).

Bottom-up: lower-level components processed first (connect_phase).

11. What is an active and passive agent:

Active agent drives stimulus (driver, monitor, sequencer).

Passive agent monitors only, for coverage and checking.

12. Why is objection used in UVM:

uvm_objection synchronizes phase completion across components, preventing premature phase termination.

13. Different ways of exiting code execution in UVM:

Objection mechanism, $finish, uvm_fatal.

14. What is TLM:

Transaction Level Modeling allows communication between components, abstracts signal-level details, and supports unidirectional, bidirectional, or broadcast connections.

15. Why use TLM_FIFO:

Stores transactions between producer and consumer with different speeds, prevents blocking, and supports methods like put(), get(), try_put(), peek().

16. Difference between uvm_sequence and uvm_sequence_item:

Sequence generates transactions, sequence_item represents a single transaction.

17. What is uvm_sequencer:

Sequencer controls the flow of sequences to the driver.

18. What is uvm_driver:

Driver converts sequence_items into pin-level or interface-level signals for the DUT.

19. What is uvm_monitor:

Monitor observes DUT transactions, collects coverage, and sends analysis data via TLM.

20. Difference between virtual interface and regular interface in UVM:

Virtual interface allows connecting a class-based testbench to DUT signals dynamically. Regular interface is static in design.

21. What is uvm_config_db:

A key-value database to configure parameters in components hierarchically.

22. Difference between pre-run and post-run phases:

Pre-run phases (build, connect) are for setup. Post-run phases (check, report, final) are for analysis and reporting.

23. What is factory override:

Allows substituting a base class object with a derived class object without changing the testbench.

24. Difference between register and field in UVM:

Registers model hardware registers; fields represent individual bits or bit-groups within a register.

25. What is uvm_subscriber:

A component that receives data via TLM for analysis or coverage collection.

26. Difference between uvm_analysis_port and uvm_analysis_export:

Port: sends transactions. Export: receives transactions. Used together for TLM communication.

27. What is uvm_scoreboard:

Component used to compare expected vs. actual DUT outputs, often using TLM for data collection.

28. Difference between uvm_report_error, uvm_report_warning, uvm_report_info:

Error: stops simulation if severe. Warning: logs a warning. Info: logs messages for debugging.

29. What is a uvm_test:

Top-level component defining the test scenario, environment, and configuration.

30. What is uvm_env:

Container for multiple components/agents, usually representing a subsystem.

31. Difference between uvm_phase and uvm_task:

Phase: defines simulation flow. Task: executes code that may consume simulation time.

32. Difference between TLM 1, 2, and 3:

TLM1: unidirectional, blocking.

TLM2: blocking and non-blocking with standard interfaces.

TLM3: advanced modeling with streaming and hierarchical connectivity.

33. What is uvm_sequence_library:

Library to register and manage sequences for reusability across tests.

34. What is uvm_constraint:

Used to define legal/randomizable transaction properties.

35. Difference between inline constraints and class constraints:

Inline: inside class variable declaration.

Class: separate constraint blocks inside class.

36. What is uvm_do_sequence:

Macro to execute a sequence on a sequencer, often with automatic start and end.

37. Difference between uvm_resource_db and uvm_config_db:

Resource DB stores shared resources across hierarchy. Config DB stores configuration parameters per instance.

38. What is uvm_callback:

A mechanism to insert custom behavior at specific points in component execution.

39. What is uvm_event:

Synchronization primitive to wait or trigger actions between components.

40. How to connect monitor to scoreboard:

Using uvm_analysis_port in monitor and uvm_analysis_export in scoreboard with TLM connection.

41. How to do coverage collection in UVM:

Define coverage classes and covergroups, connect monitors via TLM to scoreboard, and use coverage reports at the end.

42. Difference between fork/join and fork/join_none in UVM:

fork/join waits for all threads to finish. fork/join_none does not wait; threads run asynchronously.

43. Difference between constrained random and directed testing:

Constrained random uses randomization with constraints; directed testing uses fixed stimulus.

44. What is uvm_transaction:

Base class for sequence_items, supports copying, cloning, and printing.

45. What is uvm_print:

Method to display object information, often overridden for custom formatting.

46. Difference between uvm_analysis_fifo and uvm_tlm_fifo:

Both store transactions; uvm_analysis_fifo has built-in analysis connections, TLM_FIFO is more generic.

47. What is uvm_dpi:

Direct Programming Interface for connecting SystemVerilog with C/C++ models.

48. What is virtual sequencer:

Sequencer coordinating multiple sequencers in different agents.

49. What is uvm_phase_objection:

Method to keep a phase active until all components finish their execution.

50. What is sequence pre/post body:

Methods in sequence called before and after main body execution, often for setup/cleanup